

SHOPPERMATCH.AI

Complete Product, MVP Scope, Workflow & Technical Architecture Document

Prepared 2026-08-13 · Audited directly against the live codebase

Combines: Master Document · MVP Scope · Architecture · Workflows · Email & Outreach Architecture · Tools & Cost

SHOPPERMATCH AI MASTER DOCUMENT

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI

Complete Product, MVP Scope, Workflow & Technical Architecture Document

Prepared: 2026-08-13 · **Audited against:** the live codebase at `demo_tracker/` (backend FastAPI + SQLAlchemy, frontend React/TypeScript, SQLite dev DB / PostgreSQL-ready).

Companion documents:

- `SHOPPERMATCH_AI_MVP_SCOPE.md` — feature-by-feature implementation audit, user roles, roadmap
- `SHOPPERMATCH_AI_ARCHITECTURE.md` — backend/frontend/database/API/integration technical detail
- `SHOPPERMATCH_AI_WORKFLOWS.md` — every end-to-end flow as Mermaid diagrams, demo script, testing strategy
- `SHOPPERMATCH_AI_TOOLS_AND_COST.md` — technology stack, environment variables, cost estimation
- `SHOPPERMATCH_AI_EMAIL_OUTREACH_ARCHITECTURE.md` — the Email Template ↔ Outreach module in depth

This master document is the narrative entry point; the companion documents hold the exhaustive reference tables so this one stays readable end-to-end.

Verification method: every implementation claim below was checked against actual source — router decorators, SQLAlchemy model fields, `services/*.py` function bodies, frontend/src component code, `requirements.txt/package.json`, `.env.example`, `alembic/versions/`, and `docker-compose.yml`. Nothing is inferred from a UI card's label alone; where I could not verify something from source, it is marked Pending, Future, or **Not verified** rather than claimed.

Section 1 — Executive Summary

What is ShopperMatch.AI? A web application (React + FastAPI) that runs the operational core of a mystery-shopping business: creating client campaigns, defining the shops that need auditing, matching those shops against a database of mystery shoppers, sending and tracking outreach emails, and surfacing AI-generated operational recommendations that a human reviews and approves before anything is executed.

Business problem it solves. Traditional mystery-shopping recruitment is manual — a scheduler reads shopper profiles, picks candidates by memory or spreadsheet filters, sends individual emails with no delivery/open visibility, and tracks responses by hand. This doesn't scale, has no explainable "why was this shopper chosen," and provides no early warning when a campaign is at risk of missing its deadline.

Users and roles (see MVP Scope doc §4 for the verified permission boundary):

- **Admin / Campaign Manager / Scheduler** — today, a single authenticated role that does everything: creates campaigns, reviews AI recommendations, approves assignments, sends outreach, reviews analytics. (Product-

level distinctions between these three job functions exist in how the app is **used**, not as separate enforced logins yet.)

- **Client/Brand** — the conceptual beneficiary of campaign performance data; no dedicated client login exists today.
- **Shopper** — an unauthenticated external participant who receives an emailed, token-gated link and can accept/decline an assignment; never logs in.
- **AI system** — a set of stateless backend functions (`services/ai/*.py`) that compute scores, flags, and recommendations from real database rows. It never sends an email, never creates an assignment, and never modifies a record on its own — every mutating action still requires a human click through an existing approved endpoint.

Core value proposition (verified flow):

Code

```
Campaign creation → Shop management → AI-assisted shopper discovery  
→ Explainable AI matching → Human-approved assignment → Outreach  
→ Tracking → Response → Campaign analytics → AI operational recommendations
```

Each arrow above is a real, working path in the current codebase — not aspirational. See §7 for the full master flowchart.

Section 2 — Concept

See MVP Scope doc §1 for the full problem statement and the Traditional-vs-ShopperMatch.AI comparison table. In short: the original concept was to replace spreadsheet-driven shopper recruitment with a system where matching is explainable (not a black box), outreach is tracked (not "did they even see it?"), and campaign risk is visible before a deadline is missed — rather than after.

Section 3 — Updated Scope

See MVP Scope doc §2 for the full MVP / Enhanced-MVP / Future table across every functional area (recruitment, campaign management, matching, outreach, tracking, insights, integrations, AI operations, agentic AI).

Section 4 — Final MVP Scope

The complete, verified feature table (Feature / Description / User / Frontend / Backend / Database / API / Status / Dependencies) lives in **MVP Scope doc §3** — it is long enough that duplicating it here would work against readability. Headline numbers from that audit:

- **34 real API endpoints** across 13 routers, all verified by reading router source (full list in Architecture doc §5).
 - **12 database tables**, all pre-existing or added via a proper Alembic migration — no table was ever duplicated or replaced (Architecture doc §3).
 - **13 AI service modules** (`services/ai/*.py`), all local/deterministic — zero external AI API calls anywhere in this codebase.
 - **60 shopper records** in the single `shoppers` table — the demo dataset every AI feature reads from (§9 below).
-

Section 5 — Current Implementation Audit

The full Feature | Frontend | Backend | Database | Integration | Status matrix is in **MVP Scope doc §3**. Summary by area:

Area	Genuinely working?
Dashboard, Campaigns (active/upcoming/completed), Shops, Shoppers	Yes — full CRUD/read paths, real data
Recommendations (AI matching)	Yes — hard filters + weighted scoring, verified against real shopper/shop rows
Outreach, Email Templates, Invitation generation	Yes — one shared rendering service, multi-provider send
Tracking (pixel, click links, response)	Yes — real UUID tokens, real event log
Insights, Audit Logs, Settings	Yes
Integrations — SendGrid	Code path complete; defaults to mock (no external send) unless a real key is configured
Integrations — SASSIE	Demo adapter fully functional against the existing DB; real-credential path exists but unused
Integrations — Google Maps	Config/status UI only; matching uses coordinate math, not a live Maps call
Integrations — SMS	Config UI only; no send code path
Authentication	Bearer token, single admin role; no multi-role RBAC enforcement
AI Matching / "Semantic Search"	Real, but implemented as TF-cosine similarity, not a trained embedding model — this distinction is documented, not hidden
AI Scoring / Explainability	Full breakdown + confidence + reasons, all from real fields
AI Operations (Operations Engine, Action Center)	Yes — 5-agent design, verified live

Section 6 — User Roles

Full role-permission matrix in **MVP Scope doc §4**. The one fact worth stating plainly here: **the backend enforces exactly one role today**. `User.role` exists as a column but no route branches on its value — "Campaign Manager," "Scheduler," and "Client" are product-level framings of how the single Admin login is used, not separately permissioned accounts. This is stated explicitly so the document never implies RBAC exists when it doesn't.

Section 7 — Complete Application Workflow (Master Flowchart)

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart TD
    subgraph DB["Existing Database (60 shoppers, single source of truth)"]
        SHOP[(shoppers)]
        CAMP[(campaigns)]
        SHOPS[(shops)]
    end

    CAMP --> REQ[AI Requirement Parser]
```

```

REQ --> STRUCT[Structured filters]
STRUCT --> FILTER[Hard Eligibility Filtering]
SHOP --> FILTER
SHOPS --> FILTER
FILTER --> MATCH["Semantic + Structured AI Matching\n(TF-cosine + 6 weighted factors)"]
MATCH --> RANK[Explainable Candidate Ranking]
RANK --> APPROVE{Human Approval}
APPROVE -->|approve shortlist| ASSIGN[Invitation rows created]
APPROVE -->|not yet| RANK
ASSIGN --> TPL[Email Template or AI-Personalized Draft]
TPL --> RENDER[services/email.py – one shared renderer]
RENDER --> TRACK1[Tracking token + assignment link + pixel injected]
TRACK1 --> SEND{EMAIL_PROVIDER}
SEND --> SENDGRID[SendGrid / SMTP / EmailJS / mock]
SENDGRID --> OPEN["Shopper opens email\n→ tracking pixel"]
SENDGRID --> CLICK["Shopper clicks link\n→ /r/token"]
OPEN --> EVENTS[(invitation_events)]
CLICK --> EVENTS
CLICK --> LANDING[Shopper landing page]
LANDING --> RESPOND[Accept / Decline]
RESPOND --> EVENTS
EVENTS --> METRICS[Campaign progress + funnel metrics]
METRICS --> HEALTH[AI Campaign Health / Performance]
HEALTH --> OPS["AI Operations Engine\n(5 agents: coverage, outreach, deadline, shopper, campaign)"]
OPS --> CENTER[AI Action Center]
CENTER --> HUMAN2{Human Approval}
HUMAN2 -->|low-risk: view/prepare| PREP[Opens relevant tab / drafts email]
HUMAN2 -->|high-risk: assign/send/change| ASSIGN

```

Every node above maps to a verified file/endpoint — cross-reference Workflows doc §1 and Architecture doc §1/§5.

Section 8 — Campaign Workflow by Bucket

See Workflows doc §2 for the full Active / Upcoming / Completed breakdown, including which single backend function (`campaign_predictor.campaign_health`) powers both the Active-campaign health card and the Upcoming-campaign readiness card — deliberately one computation, two presentations, not two implementations.

Section 9 — Shopper Management Workflow

See Workflows doc §3. Every scored factor in AI matching (semantic similarity, distance, category experience, availability, completion history, rating, client experience) reads directly from a real `shoppers` table column — none is synthesized at request time.

Section 10 — AI Recommendation Architecture

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    REQ[Campaign/Shop requirement] --> PARSE["Requirement Parser\n(regex/keyword extraction – NOT an LLM call)"]
    PARSE --> STRUCT[Structured requirements JSON]
    STRUCT --> HARD["Hard Filters\n(active, availability, radius, min rating,\nmin completion, required categories)"]
    PROFILES[(60 shopper profiles)] --> HARD
    HARD --> ELIGIBLE[Eligible candidates]
    ELIGIBLE --> EMBED["embed() – term-frequency vector\n(explicitly a stand-in for a real embedding)"]

```

```

model)"]
  EMBED --> COSINE[Cosine similarity vs. requirement vector]
  COSINE --> STRUCT2["Structured scoring:\ndistance, category, availability,\ncompletion, rating, client history"]
  STRUCT2 --> FINAL["Final weighted score (0-100)\nsemantic 30% / distance 20% / category 15% / \navailability 15% / completion 10% / rating 5% / client 5%"]
  FINAL --> CONF["Confidence label\n(High/Medium/Low, from data completeness)"]
  FINAL --> EXPLAIN["Reasons list – only real, derived facts"]
  CONF --> OUTPUT[Ranked, explainable candidates]
  EXPLAIN --> OUTPUT

```

Important honesty note, stated explicitly in the code's own docstrings: "semantic search" here is a dependency-free term-frequency cosine similarity, **not** a trained embedding model. The function boundary (`embed()` in `services/semantic_matching.py`) is deliberately isolated so a real embedding provider could be substituted later without any caller changing. This document does not claim vector/embedding-model semantic search is implemented — it isn't.

Section 11 — AI Match Score

Full detail (weights, breakdown, confidence logic) is in Architecture doc §1 and MVP Scope doc §3. The score shown to a user (e.g. "Sarah Johnson — 85% Match, Confidence: High") is always accompanied by:

- A **breakdown table** — each of the 7 weighted factors' points/max, summing to the total.
- A **reasons list** — e.g. "4.6 km from Walmart Mumbai — Kurla," "81% completion rate," "6 previous assignments" — every line pulled from a real column, never invented.
- A **confidence label** — separate from the score itself, computed from how much real signal backs it (distance known? ≥3 prior assignments? rating present? completion rate present? prior-client history present?).

Weights are configurable in code (`MATCHING_WEIGHTS` dict in `services/semantic_matching.py`) but **not yet exposed as an admin-editable UI setting** — that's a pending refinement, not a current capability.

Section 12 — AI Features Inventory

#	Feature	Input	"Model"/service	DB dependency	Output	UI	Status	Cost	Future enhancement
1	Requirement Parser	Free-text campaign requirement	Regex/keyword extraction	<code>campaigns.parsed_requirements</code>	Structured filter JSON	Requirement ParserCard		\$0	LLM-based extraction for more flexible phrasing
2	Semantic Shopper Search	Requirement text vs. shopper profile text	TF-cosine similarity	<code>shoppers</code>	Similarity score	(component of match score)	(TF-cosine, not embeddings)	\$0	Real embedding model / pgvector

#	Feature	Input	"Model"/service	DB dependency	Output	UI	Status	Cost	Future enhancement
3	AI Shopper Recommendation	Shop + eligible shoppers	run_matching()	shoppers, shops, campaigns	Ranked candidate list	RecommendationsTab		\$0	—
4	Explainable Match Score	Same as #3	score_shopper()	same	Breakdown + reasons + confidence	CandidateCard, BreakdownModal		\$0	Admin-configurable weights
5	Candidate Ranking/Shortlisting	Same as #3	Sort + Top5/10/All toggle	same	Shortlist	RecommendationsTab		\$0	—
6	Geographic Matching	Lat/Lng of shopper + shop	haversine_km()	shoppers.latitude/longitude, shops.latitude/longitude	Distance km	CandidateCard	(coordinate math, not live Maps)	\$0	Live Google Maps routing
7	Assignment Optimization	Campaign, all shops, all eligible shoppers	Greedy hardest-shop-first optimizer	same	Proposed assignments	AutoAssignCard		\$0	Constraint-solver-based optimization
8	Acceptance Probability	Shopper's invitation history	Bayesian-smoothed estimate	invitations	Probability % or "Insufficient data"	BreakdownModal, OutreachPriorityCard		\$0	—
9	Campaign Risk/Success Prediction	Campaign, shops, shoppers, invitations	campaign_health()	multiple	Readiness %, risks	AiHealthCard		\$0	—
10	Data Quality Detection	All shoppers	Rule-based field checks	shoppers	Alerts list	DataQualityCard		\$0	—
11	Anomaly Detection	Invitations, events	Rule-based signal detection	invitations, invitation_events	Risk-flagged shoppers	AnomaliesCard		\$0	—
12	Report Summarization	Response notes	Templated aggregate summary	invitation_events.metadata.note	Executive summary	AiFeedbackCard	(scoped to notes, not a full report system)	\$0	Dedicated shopper-report table

#	Feature	Input	"Model"/service	DB dependency	Output	UI	Status	Cost	Future enhancement
13	Sentiment Analysis	Response notes	Lexicon-based	same	Sentiment + severity	AiFeedbackCard	(lexicon, not ML)	\$0	ML sentiment model
14	Report Quality Control	Response notes	Short/duplicate-note flags	same	QA flags	AiFeedbackCard	(no rating field to cross-check against)	\$0	—
15	Natural Language Insights	Free-text question	Fixed-intent keyword matcher	multiple	Answer or explicit fallback	Insights ask box		\$0	LLM-based intent parsing
16	Next Best Action	Campaign state	Rule thresholds	multiple	Action list	(superseded UI-wise by Action Center)		\$0	—
17	AI Operations Engine	Campaign state	5 agent functions	multiple	Per-campaign issues	Action Center		\$0	—
18	AI Operations Assistant	Free-text question	Same engine as #15	multiple	Answer	Dashboard chat		\$0	—
19	AI Email Personalization	Shopper, campaign, shop	Template-fill from real fields	same	Subject + body draft	Outreach "Generate with AI"		\$0	—
20	Outreach Prioritization	Match score, acceptance probability, deadline	Weighted blend	invitations, campaigns	HIGH/MEDIUM/LOW ranking	OutreachPriorityCard		\$0	—
21	Campaign Health	See #9	same function	same	Readiness breakdown	AiHealthCard		\$0	—
22	Upcoming Campaign Readiness	See #9	same function, upcoming bucket	same	Readiness breakdown	AiHealthCard (upcoming)		\$0	—
23	Completed Campaign Analysis	Invitations for a completed campaign	performance_summary()	invitations, shops	Completion/response/acceptance stats + sentence	AiPerformanceCard		\$0	—

Every "Cost" column above is \$0 because none of these features call an external paid API — this is the single most important fact for a client cost conversation and is why the Tools & Cost doc shows \$0 AI token cost at every scale tier.

Section 13 — Email Template Module (summary)

See the dedicated ``SHOPPERMATCH_AI_EMAIL_OUTREACH_ARCHITECTURE.md`` for full depth. Headline finding: the brief's core requirement — one shared backend rendering service used by both Email Templates and Outreach, no duplicated logic — **is already true today** (`services/email.py`). What's genuinely missing is a handful of `EmailTemplate` columns (`text_body`, `category`, `description`, `created_by`, `is_default`) and a template-scoped preview/test-send endpoint — both are small, additive Alembic-migration-sized gaps, not architectural rework.

Section 14 — Email Tracking Architecture (summary)

See Email/Outreach Architecture doc §4. Tracking token, pixel, and click-URL are all real UUID-based, token-gated, non-enumerable endpoints. Eleven distinct event types are recorded (a superset of the requested six), and every funnel rate (open/click/acceptance/response) is computed live from those events — never a stored, staleness-prone percentage.

Section 15 — Outreach Architecture (summary)

See Email/Outreach Architecture doc §5 and Workflows doc §4. Retry (up to 3 attempts), failure surfacing (`email_jobs.last_error` + an `invitation_events` row), and audit logging are all real, verified code paths.

Section 16 — Backend Architecture

See Architecture doc §1. Stack: FastAPI + SQLAlchemy 2.0 async + Alembic, no separate repository layer (routers call service functions directly), background email delivery via an in-process `asyncio` worker (no Celery/Redis).

Section 17 — Frontend Architecture

See Architecture doc §2. React 18 + TypeScript + Tailwind + Recharts, no state-management library beyond a small custom `useApi` hook, single-origin deployment (FastAPI serves the built SPA).

Section 18 — Database Architecture

See Architecture doc §3 for the full table list and ER diagram. 12 tables, all pre-existing or added via a documented Alembic migration; no table was ever duplicated.

Section 19 — 50(+)-Member Demo Database

See Architecture doc §4. **Verified: 60 rows currently in the `shoppers` table** (the original 50-record seed plus a later 10-record batch — never a second table). Every AI feature listed in §12 above reads this same table; none has ever written a parallel dataset.

Section 20 — API Documentation

The full endpoint table (34 real endpoints, method/path/auth/purpose) is in Architecture doc §5. No endpoint listed there is **PROPOSED** — every one exists in the current router source.

Section 21 — Integration Architecture

See Architecture doc §6. Verified status per integration: SendGrid (code complete, defaults to mock), SASSIE (demo adapter live, real credentials never supplied), Google Maps (config only, not called by matching), SMS (config only, no send path), external AI provider (not used anywhere).

Section 22 — Technology Stack

Full table in Tools & Cost doc §4.

Section 23 — Tools, Services, API Keys & Cost ("Tab 5")

Full detail in `SHOPPERMATCH_AI_TOOLS_AND_COST.md`. The single most quotable fact: **this application currently runs with \$0 external API dependency** — email defaults to a mock provider, AI is 100% local computation, Maps/SMS/SASSIE all have working demo fallbacks. Every dollar figure in the cost doc is explicitly caveated as approximate and marked for verification against current official pricing — nothing is presented as a confirmed, current number.

Section 24 — Environment Variables

Full table in Tools & Cost doc §5 — variable names and purpose only, no secret values reproduced anywhere in this documentation set.

Section 25 — Security Architecture

Control	Status	Detail
Authentication		Bearer token (HS-signed via SECRET_KEY), 12-hour default expiry
Authorization / RBAC		Single role enforced; users.role column unused for branching
API key protection		IntegrationConfig.secret_config is a separate column, verified never included in any API response (serializers.py)
Environment variables		All secrets sourced from env vars / .env, never hardcoded; .env is gitignored

Control	Status	Detail
Database security		No row-level security; single-tenant assumption (no multi-client data isolation yet)
Email security		Optional SendGrid webhook ECDSA signature verification; HTML sanitization strips script/iframe/on*-attribute/javascript: constructs on admin-authored template bodies
Tracking security		UUIDv4 tokens (non-guessable), public endpoints only resolve the exact token given, rate-limited (TRACKING_RATE_LIMIT_PER_MINUTE)
PII protection		No field-level encryption at rest; standard practice for a demo, would need review before handling real PII at scale
Rate limiting		Present on tracking endpoints only; not on general API routes
Input validation		Pydantic models on every request body
SQL injection protection		100% SQLAlchemy ORM/Core parameterized queries; the NL Insights feature explicitly never executes free-text SQL
XSS protection		Template HTML sanitization is a best-effort regex strip, explicitly documented in code as "not a substitute for a real sanitizer library (e.g. bleach)"
Audit logs		audit_logs table, populated for every AI action and key admin action

Section 26 — Deployment Architecture

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    DEV["Local development\nnpm run dev + uvicorn --reload"] --> STAGING
    subgraph STAGING ["Staging / Demo"]
        DOCKERIMG["Single Docker image\n(Dockerfile: builds React, bundles into FastAPI static/)"]
    end
    STAGING --> PROD
    subgraph PROD ["Production"]
        HOST["Render / Railway / equivalent"] --> APP["FastAPI + built SPA, one process"]
        DBHOST["Managed PostgreSQL"] --> APP
        APP --> SENDGRID["SendGrid — real key"]
        APP --> TRACK["Tracking endpoints — same app"]
    end

```

Verified: `docker-compose.yml` already defines a two-service stack (Postgres 16 + the app image) that runs with `docker compose up --build` — this is a real, working deployment path today, not a proposal. What's *not* yet done is actually deploying that image to a hosting provider; the compose file is a local/self-hosted deployment target.

Section 27 — Testing Strategy

See Workflows doc §8. Verified: one test file exists (backend/tests/test_acceptance.py). The full proposed E2E scenario (50 shoppers → campaign → AI matching → outreach → SendGrid → tracking → acceptance → analytics) is documented step-by-step there and flagged clearly as not yet implemented as an automated test.

Section 28 — Demo Scenario

Full 14-step client demo script in Workflows doc §7, built around features that are actually live (no step requires anything hypothetical).

Section 29 — Current vs. Future Architecture

Current (verified):

Code

```
React → FastAPI → SQLite (dev) / PostgreSQL (compose-ready) → local AI services → SendGrid/mock → tracking (same process)
```

Future (proposed, not built):

Code

```
React → API Gateway → FastAPI services → Event Bus → AI Agent Orchestrator  
→ multiple specialized agents → PostgreSQL + Vector DB + (optionally) Neo4j → external integrations (live SASSIE, live Maps, SMS)
```

Why each future technology is optional, not urgent (full reasoning in Architecture doc §7):

- **Event bus / Kafka** — nothing currently needs asynchronous cross-service event propagation; the existing outbox worker + on-demand computation cover every present use case.
 - **Vector DB** — TF-cosine over 60 records is sub-millisecond; a vector index has no measurable benefit yet.
 - **Neo4j** — no relationship/graph query has been requested that SQL can't already answer at this data volume.
 - **Agent orchestration framework (LangGraph, etc.)** — the current 5-agent Operations Engine already separates responsibilities cleanly with plain async functions; a framework would change plumbing, not capability.
 - **API Gateway** — meaningful once there are multiple backend services; today there is one FastAPI app.
-

Section 30 — Agentic AI Roadmap

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart LR
    MON[Monitor] --> DET[Detect] --> AN[Analyze] --> REC[Recommend] --> APP[Request Approval] --> EXE[Execute — low-risk only] --> TRK[Track] --> LEARN[Learn]
```

Distinguishing three terms precisely, per the request:

Term	What it means	What exists today
AI Assistant	Answers questions on demand, no autonomous action	Implemented — <code>insights_agent.answer_question()</code> , the Operations Assistant chat box
AI Agent	A bounded function with a specific responsibility that can <i>*propose*</i> an action	Implemented — the 5 Operations Engine agents (coverage/outreach/deadline/shopper/campaign), each single-purpose
Agentic Workflow	Multiple agents chained with a monitor→detect→analyze→recommend→approve→execute→track loop, where "execute" can include autonomous low-risk steps	Partially implemented — the monitor→detect→analyze→recommend→approve chain is real (Operations Engine → Action Center → human click); the "execute" step is always a human-triggered call to an existing endpoint today, never autonomous. Full agentic autonomy (auto-executing low-risk actions like sending a reminder without a click) is proposed, not built.

Future agents beyond the current 5 (Campaign Monitoring, Shopper Matching, Coverage, Outreach, Response, Risk, Data Quality, Operations — from the roadmap brief): most already exist as functions today (Shopper Matching = `semantic_matching.py`, Risk = `anomaly_detector.py`, Data Quality = `data_quality.py`, Coverage/Outreach = Operations Engine agents). What's genuinely new in a future phase is formalizing them as message-passing agents in an orchestration framework and allowing explicitly-scoped low-risk actions (generate reminder, prepare email, suggest candidates) to execute without a per-instance click — while keeping assign/send/bonus/parameter-change actions human-gated permanently.

Section 31 — Feature Status Roadmap

Full Phase 1–4 table in MVP Scope doc §5.

Section 32 — Risks & Limitations

Risk	Mitigation already in place	Residual risk
AI hallucination	Every AI feature is deterministic and reads only real DB fields; NL Insights has a fixed intent list + explicit fallback string; acceptance probability refuses to guess below a data threshold	Low, but a future LLM-based upgrade would reintroduce this risk and need new guardrails
Insufficient shopper history	Acceptance predictor explicitly returns "Insufficient historical data" rather than a fabricated number	Reduces trust in early-stage campaigns with few responses — inherent to the demo dataset size
Embedding quality	TF-cosine is a simpler, less semantically rich signal than a trained embedding model	Documented explicitly, not hidden; upgrade path is isolated behind <code>embed()</code>
Prediction accuracy	Campaign readiness/health scores are heuristic blends, not validated against historical ground truth	No backtesting has been done — should be flagged to any client relying on these for hard decisions
Email deliverability	Multi-provider support, retry logic, webhook-based bounce tracking	Real-world deliverability (SPF/DKIM/domain reputation) is outside this app's control

Risk	Mitigation already in place	Residual risk
Tracking limitations	Pixel-based open tracking is blocked by some email clients' image-proxying/caching (a known industry-wide limitation, not a bug here)	Open-rate numbers may undercount
Third-party API failures	SendGrid/Maps/SASSIE all have documented fallback behavior (mock/coordinate-math/demo adapter)	A real production dependency on any of these still needs its own monitoring
SASSIE dependency	Demo adapter fully decouples current features from real SASSIE	Real-world data quality from SASSIE, once connected, is unverified
Google Maps dependency	Coordinate math fallback already covers the core matching use case	Live traffic/routing-aware distance is not available without a real key
AI cost	\$0 today — no external AI calls	Would need re-evaluation only if a future phase adopts a paid AI provider
Privacy	No field-level PII encryption at rest	Needs review before handling real (non-demo) shopper PII at scale
Scalability	SQLite is single-writer; Postgres path is compose-ready but unproven at real load; TF-cosine matching is $O(n)$ over shoppers per request (fine at 60, needs revisiting well beyond that)	Should be load-tested before a production commitment

Section 33 — Final System Workflow

See Section 7 above — the master Mermaid flowchart already covers database, campaign, shop, AI matching, recommendation, assignment, email template, outreach, SendGrid, tracking, response, analytics, AI operations, and human approval in one diagram.

Section 34 — Final Client-Facing Summary

What ShopperMatch.AI does. It's the system ISN uses to run mystery-shopping campaigns end-to-end: set up a campaign and the shops that need visiting, let the AI suggest which of your mystery shoppers are the best fit and explain exactly why, send them a personalized invitation, watch in real time whether they opened it, clicked it, and accepted — then see AI-generated recommendations about what needs attention next.

Why it's valuable. It replaces a spreadsheet-and-memory process with something explainable and trackable. Nobody has to guess why a shopper was picked — the system shows the match score breakdown. Nobody has to wonder if an email was even seen — it's tracked. And instead of a manager discovering a staffing gap the week before a deadline, the AI Action Center surfaces it as soon as the pattern appears.

How AI is used. Every AI feature in this app today runs on the business's own data, locally, with no per-use cost — it parses plain-English requirements into filters, ranks shoppers with a transparent scorecard (never a black-box number), estimates how likely a shopper is to accept (and says so honestly when there isn't enough history to estimate), drafts personalized outreach emails a human still reviews before sending, and watches every active campaign for coverage gaps, stalled outreach, and looming deadlines.

How outreach works. Pick a campaign, shop, and shopper — or let the AI's outreach-priority list tell you who to contact first. Use a saved template or generate a draft with AI. Review and edit it. Send it. The system automatically embeds a tracking link and open pixel so you can see exactly what happens next.

How tracking works. Every invitation gets a unique, unguessable link. When it's opened, clicked, accepted, or declined, that's recorded immediately and rolls up into campaign-level funnel metrics (open rate, click rate, acceptance rate) — all computed live, never a stale number.

How campaigns are managed. Active campaigns show live shop coverage and outreach progress. Upcoming campaigns show AI readiness — is there enough shopper coverage before it even starts? Completed campaigns get an AI performance summary comparing cities, response rates, and acceptance patterns.

What's currently available. Everything described above is live and has been verified against the real application — not a mockup. See MVP Scope doc §3 for the itemized, audited list.

What's coming next. Formal multi-role permissions (Campaign Manager / Scheduler / Client logins with distinct access), a real embedding-based matching upgrade if match quality needs to go beyond keyword-level semantic similarity, live Google Maps routing, an SMS outreach channel, and — further out — an agentic layer that can execute clearly low-risk actions (like sending a reminder) without a manual click, while anything that assigns a shopper, sends real outreach, or changes campaign parameters stays human-approved permanently, by design.

SHOPPERMATCH AI MVP SCOPE

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI — MVP Scope & Implementation Audit

Companion document to `SHOPPERMATCH\_AI\_MASTER\_DOCUMENT.md`. Audited directly against the codebase at C:\Users\iSN-037\Desktop\demo_tracker on **2026-08-13**. Every status below was verified by reading the actual router/service/model source — nothing here is inferred from UI screenshots or naming alone.

Status legend: Implemented · Partial · Pending · Future/Proposed

1. Concept

Problem statement. Mystery-shopping operations (ISN) recruit shoppers for retail audit assignments manually: campaign managers eyeball a shopper spreadsheet, pick candidates by memory or ad hoc filters, email them individually, and track responses in a separate sheet. This doesn't scale past a handful of campaigns and has no attribution back to **why** a shopper was chosen or **whether** an email was even opened.

Target users. ISN campaign managers/schedulers (internal), mystery shoppers (external, unauthenticated), and — in the documentation's business framing — the client brands ISN runs mystery-shopping audits for.

Business objective. Replace manual shopper selection and outreach with a system that (a) matches shoppers to shop requirements explainably, (b) sends and tracks outreach end-to-end, and (c) surfaces AI-generated operational recommendations a human still has to approve.

Core workflow. Campaign → Shop → AI-recommended shoppers → human-approved outreach → tracked response → campaign analytics.

Expected outcome. Less manual shopper triage, an auditable trail from "why was this shopper picked" to "did they open the email, click it, accept," and a single system of record instead of spreadsheets + email + a separate tracker.

Traditional vs. ShopperMatch.AI workflow

Step	Traditional mystery-shopping workflow	ShopperMatch.AI workflow
Shopper selection	Manual, spreadsheet/memory-based	AI hard-filter + weighted match score, explainable
Outreach	Individual emails, no tracking	Templated + AI-personalized, UUID-tracked
Delivery visibility	None (no idea if it was read)	Pixel-based open tracking, click tracking
Response capture	Phone/email replies, manually logged	Self-service accept/decline landing page, timestamped

Step	Traditional mystery-shopping workflow	ShopperMatch.AI workflow
Campaign status	Manually maintained	Computed from real invitation/shop data
Recruitment gap detection	Discovered late, reactively	AI Operations Engine flags gaps proactively
Audit trail	Rare/inconsistent	Every AI action + admin action logged to audit_logs

2. Updated Scope

Area	MVP	Enhanced MVP	Future / full-scale
Shopper recruitment	Static DB of demo shoppers	SASSIE demo-adapter sync	Real SASSIE production sync
Campaign management	CRUD + status buckets	AI requirement parsing on campaigns	Multi-client permissioning
Shop management	CRUD, coordinates, requirements	Geographic matching	Google Maps live routing
Shopper recommendation	Rule-based scoring	Semantic (TF-cosine) + hard filters	Real embeddings / vector DB
Outreach	Manual template send	AI-personalized email generation	Multi-channel (SMS) send
Email templates	CRUD, variable substitution	Shared render service w/ Outreach	Categories, per-client defaults
Tracking	Pixel + click token	Funnel analytics dashboard	Deliverability/bounce intelligence
Insights	Static rule-based cards	AI Natural Language Insights	Predictive trend analysis
Audit logs	Present, admin actions	AI-action logging	Exportable compliance reports
Integrations	Config/status UI	SASSIE demo adapter, SendGrid	Live SASSIE, live Maps, SMS
AI operations	—	Operations Engine + Action Center	Autonomous low-risk execution
AI analytics	—	Campaign health/readiness/performance	Trained predictive models
Agentic AI	—	Rule-based multi-agent pipeline (no framework)	LangGraph/orchestrated agents

3. Final MVP Scope (feature-by-feature, verified)

Feature	Description	User	Frontend	Backend	DB	API	Status	Dependencies
Auth	Bearer-token login, single admin role	Admin	Login.tsx	routers/auth.py	users	POST /api/auth/login, GET /api/auth/me		—
Campaign CRUD/listing	List/detail/bucket (active/upcoming/completed)	Admin	Campaigns.tsx, CampaignsPortal.tsx, CampaignDetail.tsx	routers/campaigns.py	campaigns, shops	GET /api/campaigns, GET /api/campaigns/{id}		—
Shop management	Per-campaign shop list w/ coordinates, requirements	Admin	Shops.tsx, Campaign "Shops" tab	routers/shops.py	shops	GET /api/shops, GET /api/shops/{id}		Campaign
Shopper directory	List/detail/campaign history	Admin	Shoppers.tsx, ShopperDrawer.tsx	routers/shoppers.py	shoppers	GET /api/shoppers, GET /api/shoppers/{id}		—
AI requirement parser	Regex/keyword extraction of NL campaign requirements into structured filters	Admin	RequirementParserCard (Campaign Insights tab)	services/ai/requirement_parser.py	campaigns.parsed_requirements (JSON)	POST /api/ai/parse-requirements		—
AI shopper matching	Hard-filter → TF-cosine "semantic" score → weighted structured score	Admin	RecommendationSTab (Campaign detail)	services/semantic_matching.py	shoppers, shops, campaigns	GET /api/campaigns/{id}/shops/{id}/recommendations		Shopper data completeness
Explainable match score	Score breakdown + confidence + reasons, all from real fields	Admin	CandidateCard, BreakdownModal	services/semantic_matching.py::score_shopper	same	(part of recommendations response)		—
Candidate shortlist	Top 5 / Top 10 / All Eligible, select, approve, view profile	Admin	RecommendationSTab	routers/campaigns.py::approve_ai_recommendations	invitations created on approval	POST .../recommendations/approve		—
Assignment optimization	Greedy multi-shop optimizer, review-then-approve	Admin	AutoAssignCard	services/ai/assignment_optimizer.py	invitations on approval	POST /api/ai/campaigns/{id}/optimize-assignments		Approve endpoint above
Acceptance probability	Bayesian-smoothed estimate from real invitation history; explicit "insufficient data" fallback	Admin	BreakdownModal, OutreachPriorityCard	services/ai/acceptance_predictor.py	invitations	GET /api/ai/acceptance-probability		≥3 historical responses per shopper for a number

Feature	Description	User	Frontend	Backend	DB	API	Status	Dependencies
								r (else null)
Outreach prioritization	HIGH/MEDIUM/LOW ranking combining match score, acceptance probability, deadline urgency	Admin	OutreachPriorityCard	services/ai/outreach_priority.py	—	GET /api/ai/campaigns/{id}/shops/{id}/outreach-priority		Acceptance predictor, matching
Campaign health/readiness/performance	One scoring function, 3 presentations by campaign bucket	Admin	AiHealthCard, AiPerformanceCard	services/ai/campaign_predictor.py	—	GET /api/ai/campaigns/{id}/health,/performance		Matching engine
Data quality agent	Missing experience/location/invalid email/duplicate detection	Admin	Insights page DataQualityCard	services/ai/data_quality.py	shoppers	GET /api/ai/data-quality		Read-only, never auto-fixes
Anomaly/fraud risk detection	Location inconsistency, unusually fast responses, repeated timing, perfect acceptance rate	Admin	Insights page AnomaliesCard	services/ai/anomaly_detector.py	invitations, invitation_events	GET /api/ai/anomalies		"Potential Anomaly," never "fraudulent"
Report summarization / sentiment / QA	Lexicon sentiment + executive summary + short/duplicate-note QA flags, scoped to InvitationEvent.note	Admin	AiFeedbackCard	services/ai/report_analysis.py	invitation_events.meta data.note	GET /api/ai/campaigns/{id}/feedback-analysis		No dedicated shopper-report table exists — scoped to response notes only
Natural language insights / Operations Assistant	Fixed-intent keyword matcher over safe read-only queries; explicit fallback string	Admin	Insights ask box, Dashboard chat	services/ai/insights_agent.py	reads multiple tables	POST /api/ai/ask		No free-text SQL ever executed
Next-best-action	Per-campaign flags (coverage/response/completion)	Admin	(superseeded on Dashboard by Action Center;	services/ai/next_best_action.py	—	GET /api/ai/next-best-actions		—

Feature	Description	User	Frontend	Backend	DB	API	Status	Dependencies
			endpoint still live)					
AI Operations Engine	5 agents (coverage/outreach/deadline/shopper/campaign) → per-campaign top issue	Admin	ActionCenterCard	services/ai/operations_engine.py	—	GET /api/ai/action-center		campaign_predictor
Integration awareness	Honest notices when SASSIE/Email/Maps/SMS aren't fully connected	Admin	Action Center banner	services/ai/integration_awareness.py	integration_configs, sync_logs	(bundled in /api/ai/action-center)		—
AI email personalization	Generates subject/body from real shopper/campaign/shop fields; never auto-sends	Admin	Outreach "✦" Generate with AI"	services/ai/email_personalizer.py	—	POST /api/ai/personalize-email		Human must still click Send
Email templates	Create/edit/duplicate/delete, variable tokens	Admin	Outreach.h.tsx template dropdown	routers/email_templates.py	email_templates	GET/POST/PUT/DELETE /api/email-templates*		See email doc for gaps (no plain-text body, category, is_default)
Outreach send	Template/AI-composed → tracked invitation → SendGrid/mock/SMTP/EmailJS/direct-MTA	Admin	Outreach.h.tsx	routers/invitations.py, services/email.py, services/outbox.py	invitations, email_jobs, email_compositions	POST /api/invitations, /send, /send-test		Email provider config
Tracking (pixel + click + response)	1x1 gif open pixel, /r/{token} click redirect, self-service accept/decline page	Admin + Shopper	Tracking.tsx, ShopperInvite.tsx	routers/tracking.py	invitation_events	GET /r/{token}, GET /track/open/{token}.gif, POST /api/invitations/{token}/respond		—
Dashboard	Funnel KPIs, Action Center, Operations Assistant, recent activity	Admin	Dashboard.tsx	routers/dashboard.py	multiple	GET /api/dashboard/metrics		—
Audit logs	Every recorded AI + admin action	Admin	AuditLogs.tsx	services/audit.py, routers/misc.py	audit_logs	GET /api/audit-logs		—

Feature	Description	User	Frontend	Backend	DB	API	Status	Dependencies
Integration hub	SASSIE (demo adapter)/Email/Maps/SMS/AI config + test + status	Admin	Integrations.tsx	routers/integrations.py	integration_configs, sync_logs	GET/PUT/POST /api/integrations/*	for SASSIE demo + Email; Maps/SMS (config UI exists, no live API call implemented)	—
Notifications	Recent event feed / bell dropdown	Admin	NotificationsBell.tsx	routers/notifications.py	invitation_events	GET /api/notifications		—
RBAC (multi-role)	Campaign Manager / Scheduler / Client distinct permissions	Admin only today	—	deps.py::get_current_user (single check, no role branching)	users.role column exists but unused for authorization	—		users.role field exists but every route only checks "is authenticated," not role
Real vector embeddings	sentence-transformers / pgvector	—	—	—	—	—		Documented as a deliberate scope decision, not a gap — see Architecture doc
SASSIE live production sync	Real SASSIE API credentials	—	Integrations page already	services/integrations/sassie.py has both a demo adapter and a real-credential code path	integration_configs	POST /api/integrations/sassie/sync		Adapter exists; real

Feature	Description	User	Frontend	Backend	DB	API	Status	Dependencies
			has the UI					credentials never supplied in this project
Google Maps live routing	Real geocoding/distance via Maps API	—	Integrations page has config UI	haversine_km (coordinate math) is what recommendations actually use	—	POST /api/integrations/maps/test (config test only)		Maps key never configured; distance always coordinate-based
SMS outreach	Send via SMS provider	—	Integrations page has config UI	No SMS-send code path exists	—	—		No provider integration written

4. User Roles

Ground truth: the backend has exactly one authenticated role today (`User.role` defaults to "admin"; no route branches on it). The roles below are the *product/business* roles the application's workflow implies — only "Admin" is currently enforced in code. Everything else is a documentation framing for future RBAC, not a current permission boundary.

Role	Status	Pages accessible	Actions allowed	AI capabilities	Approval requirements
Admin (ISN operator)	Implemented (the only real role)	All	Everything below	All AI features	Approves all AI-suggested actions
Campaign Manager	Conceptual (same login as Admin today)	Same as Admin	Create/manage campaigns, shops, outreach	Requirement parsing, recommendations, action center	Approves recommendations/assignments
Scheduler	Conceptual	Same as Admin	Outreach scheduling, follow-ups	Outreach prioritization, email personalization	Approves sends
Client/Brand admin	Conceptual — no client-scoped login exists	None dedicated today	Would view campaign performance only	Campaign health/performance summaries	Read-only in concept

Role	Status	Pages accessible	Actions allowed	AI capabilities	Approval requirements
Shopper	Implemented, but unauthenticated	/invite/{token} public landing page only	Accept/decline, optional note	None	N/A — self-service
AI system	Implemented as a set of stateless services, not a login	N/A	Computes recommendations/scores/flags; never sends, assigns, or modifies data on its own	See AI Features Inventory (Master doc §12)	Every mutating action requires a human click through an existing approved endpoint

Role-permission matrix (as actually enforced today)

Capability	Admin	Shopper (public link)
View/create campaigns, shops	✓	✗
Run AI matching / view recommendations	✓	✗
Approve recommendations → create invitations	✓	✗
Generate/send outreach email	✓	✗
View tracking/analytics/audit logs	✓	✗
Accept/decline own invitation	✗	✓(via unguessable token only)
Configure integrations	✓	✗

5. Feature Status Roadmap

Phase 1 — Core MVP (done)

Campaign/Shop/Shopper CRUD, auth, outreach send, tracking pixel/click/response, email templates, audit logs, dashboard KPIs.

Phase 2 — AI Enhancement (done)

Requirement parser, hard-filter + semantic matching, explainable score + confidence, shortlist tiers, acceptance probability, email personalization, campaign health/readiness/performance.

Phase 3 — Intelligent Operations (done)

Operations Engine (5 agents) + Action Center, outreach prioritization, data quality agent, anomaly detection, NL insights/Operations Assistant, integration awareness.

Phase 4 — Agentic Platform (future)

Item	Priority	Dependencies	Complexity	Tools	Cost impact
Multi-role RBAC	High	<code>users.role</code> already exists	Medium	None new	None
Real embeddings / vector search	Medium	Embedding provider or local model	Medium	sentence-transformers (free) or API embeddings (paid)	Low–medium
Autonomous low-risk execution (e.g. auto-send reminders)	Medium	Operations Engine (done) + explicit opt-in	Medium	None new	None
Orchestrated multi-agent framework (LangGraph or similar)	Low	Only if genuinely needed — current 5-function agent design already separates responsibilities	High	LangGraph (free, OSS)	Low (self-hosted)
Live SASSIE / Maps / SMS	High (for production)	Real vendor credentials	Low (adapters already exist)	Vendor accounts	See Tools & Cost doc
Neo4j graph layer	Low	Only if regional-density queries become a real need	High	Neo4j (free community / paid Aura)	Medium if adopted
Event bus (Kafka)	Low	Only past a scale where polling/webhooks are insufficient	High	Kafka or managed equivalent	Medium if adopted

See `SHOPPERMATCH_AI_MASTER_DOCUMENT.md` for narrative context, `SHOPPERMATCH_AI_ARCHITECTURE.md` for technical depth.

SHOPPERMATCH AI ARCHITECTURE

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI — Technical Architecture

Verified against the codebase on 2026-08-13. All facts (versions, files, endpoints) were read directly from source — nothing is assumed.

1. Backend Architecture

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart TD
    FE[React SPA] -->|fetch, bearer token| API[FastAPI app.main]
    API --> R1[routers/*.py]
    R1 --> SVC[services/*.py]
    SVC --> AISVC[services/ai/*.py]
    SVC --> DB[(SQLAlchemy async ORM)]
    DB --> PG[(SQLite file - dev/demo)]
    DB -->|DATABASE_URL swap, no code change| PG2[(PostgreSQL - docker-compose)]
    SVC --> EMAIL[services/email.py]
    EMAIL --> SG[SendGrid / SMTP / EmailJS / direct-MTA / mock]
    SVC --> OUTBOX[services/outbox.py background worker]
    OUTBOX --> EMAIL
```

Actual stack (from backend/requirements.txt): FastAPI 0.115.6, Uvicorn 0.32.1, SQLAlchemy 2.0.36 (async), asyncpg 0.30.0, aiosqlite 0.20.0, Pydantic 2.10.4, Alembic 1.14.0, cryptography 50.0.0, httpx 0.28.1 (used for outbound HTTP to SendGrid/EmailJS/SASSIE-demo, not a Python SDK for any of them). No ORM "repository" layer exists — routers call service functions that use SQLAlchemy directly; there is no separate repository abstraction layer.

AI services are plain async Python functions in backend/app/services/ai/*.py — there is **no embedding service, no vector search service, and no external AI API call anywhere in this codebase**. services/semantic_matching.py implements a dependency-free term-frequency cosine-similarity function (embed()/_cosine()) as an explicitly-labeled stand-in for a real embedding model; the function boundary is deliberate so a real embedding provider could replace it later without touching any caller.

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart TD
    REQ[Campaign/Shop requirement text] --> PARSER[requirement_parser.py - regex/keyword extraction]
    PARSER --> STRUCT[Structured filters JSON]
    STRUCT --> FILTER[semantic_matching.run_matching - hard filters]
    SHOPPERS[(shoppers table, 60 rows)] --> FILTER
    FILTER --> SCORE[score_shopper - TF-cosine + 6 structured factors]
    SCORE --> RESULT[match_score, confidence, breakdown, reasons]
```

Email architecture:

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    OUTREACH[Outreach.tsx composer] --> API1[POST /api/invitations]
    API1 --> EMAILSVC[services/email.py]
    EMAILSVC --> CTX[build_variable_context]
    CTX --> RENDER[render_composed_email / render_email]
    RENDER --> SEND[send_email - provider dispatch]
    SEND -->|EMAIL_PROVIDER=sendgrid| SENDGRID[SendGrid HTTP API via httpx]
    SEND -->|EMAIL_PROVIDER=smtp| SMTP[Gmail/SMTP AUTH]
    SEND -->|EMAIL_PROVIDER=emailjs| EMAILJS[EmailJS REST API]
    SEND -->|EMAIL_PROVIDER=direct| MTA[Direct-to-MX delivery]
    SEND -->|EMAIL_PROVIDER=mock - default| MOCK[No external call, returns rendered message]

```

Tracking architecture:

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    EMAIL[Sent email] --> PIXEL["/track/open/{token}.gif"]
    EMAIL --> LINK["/r/{token}?utm_*"]
    PIXEL --> TRACKSVC[routers/tracking.py]
    LINK --> TRACKSVC
    TRACKSVC --> EVENTS[(invitation_events)]
    LINK --> REDIRECT[302 to shopper landing /invite/token]
    REDIRECT --> RESPOND["POST /api/invitations/{token}/respond"]
    RESPOND --> EVENTS
    EVENTS --> DASH[Dashboard/Tracking analytics]

```

2. Frontend Architecture

Verified from `frontend/package.json`: **React 18.3.1, TypeScript 5.5.4, react-router-dom 6.26.2, Tailwind CSS 3.4.13, Recharts 2.12.7, Vite 5.4.8**. No Redux/Zustand/React Query — state is local `useState` plus one custom hook (`lib/useApi.ts`) providing `{data, loading, error, reload}`. No axios — `lib/api.ts` wraps native `fetch` with a bearer-token header and typed helper methods per endpoint.

Pages (`frontend/src/pages/`): Home, Login, Dashboard, Campaigns, CampaignsPortal, CampaignDetail (largest file, ~52 KB — houses Overview/Shops/Shoppers/AI Recommendations/Outreach/Tracking/Insights/Audit Logs tabs), Shops, Shoppers, Recommendations, Outreach (~39 KB), Tracking, Insights, AuditLogs, Integrations, Settings, ShopperInvite (the public unauthenticated landing page).

Components (`frontend/src/components/`): Layout (nav shell), Icons (inline SVG set, no icon package dependency), `ui.tsx` (Badge/Spinner/Loading/KpiCard/useToast), Funnel, Timeline, Attribution, InvitationDrawer, ShopperDrawer, NotificationsBell.

Routing: client-side via `react-router-dom`; the FastAPI backend serves the built SPA and falls back to `index.html` for any non-`/api`, non-`/r`, non-`/track` path (see `main.py::spa`), so the whole app is single-origin.

Error handling: every data-fetching call goes through `lib/api.ts::request()`, which throws a typed `ApiError` (status + message); pages render an `ErrorBox` component with retry. 401 responses clear the stored token automatically.

3. Database Architecture

Engine (verified): `sqlite+aiosqlite:///./shoppermatch.db` by default (`backend/app/config.py`). `DATABASE_URL` supports a Postgres async URL (`postgresql+asyncpg://...`) with zero code changes — the app's JSON columns use `JSON().with_variant(JSONB(), "postgresql")` and Alembic migrations use `render_as_batch` specifically so they

work on both engines. ``docker-compose.yml`` provisions a real Postgres 16 container for the containerized deployment path, but the environment this was audited in runs the SQLite file directly via `uvicorn`. Both are the **same schema** — there is no separate demo-vs-production data model.

Tables (all existing — read from `backend/app/models.py`)

Table	Purpose	Key fields
users	Admin login	id, name, email, role, password_hash
shoppers	The 60-record demo shopper pool	see §4
campaigns	Client campaigns	id, name, client_name, status, total/completed/remaining_shops, deadline, requirements_text, parsed_requirements (JSON), source, external_id
shops	Physical locations within a campaign	id, campaign_id, coordinates, required_shoppers, compensation, category, visit_start/end, source, external_id
invitations	One outreach message → one shopper → one shop	tracking_token (UUID), reference, status, sent/delivered/opened/clicked/responded timestamps, response, UTM fields
invitation_events	Immutable event log per invitation	event_type, event_timestamp, metadata (JSON, Python attr event_metadata)
email_jobs	Outbox queue + retry history	provider, status, attempts, last_error, next_attempt_at
email_templates	Reusable subject/body templates	name, subject, html_body, active
email_compositions	Per-invitation hand-edited subject/body override	invitation_id (unique), subject_template, html_template
integration_configs	SASSIE/Email/Maps/AI/SMS provider config + status	provider, status, configuration (non-secret JSON), secret_config (JSON, never serialized to any API response)
sync_logs	One row per SASSIE sync run	provider, status, campaigns/shops/shoppers fetched/created/updated, errors
audit_logs	Every logged admin/AI action	actor, action, entity_type, entity_id, summary, meta

ER diagram

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
erDiagram
    CAMPAIGNS ||--o{ SHOPS : contains
    CAMPAIGNS ||--o{ INVITATIONS : has
    SHOPS ||--o{ INVITATIONS : targets
    SHOPPERS ||--o{ INVITATIONS : receives
    INVITATIONS ||--o{ INVITATION_EVENTS : logs
    INVITATIONS ||--o{ EMAIL_JOBS : queues
    INVITATIONS ||--o{ EMAIL_COMPOSITIONS : "may override"
    USERS ||--o{ AUDIT_LOGS : performs
```

Labeling per the request's requirement — existing vs. proposed:

- **Existing:** every table above.

- **Proposed / not present:** a dedicated `recommendations` table (recommendation results are computed on demand, never persisted — a deliberate choice, see §7), `ai_alerts/ai_action_records` tables (Operations Engine output is also computed live, not persisted — same reasoning), `assignments` as a distinct table (assignment *is* an `Invitation` row in this schema; there is no separate assignment entity).

4. The 50(+)-Member Demo Database

Verified count: the `shoppers` table currently holds **60 rows** — the original 50-member seed plus a 10-shopper batch added in a later session turn. This is still the single existing dataset; no second table or file was ever created to hold shopper records.

Fields actually populated (from `models.py` + migrations 0001–0003): `shopper_code`, `name`, `email`, `phone`, `city`, `state`, `zip_code`, `latitude`, `longitude`, `categories` (JSON list), `availability_status`, `source`, `rating`, `completion_rate`, `previous_assignments`, `active`, `gender`, `age`, `pincode`, `skills`, `experience_description`, `years_experience`, `preferred_distance_km`, `preferred_locations`, `preferred_categories`, `languages`, `certifications`, `previous_clients`, `updated_at`, `external_id`.

Every AI feature reads from this one table — requirement parsing produces filters that are applied to it, matching scores it, the acceptance predictor reads `invitations` joined back to it, data quality/anomaly detection scan it directly, and the Operations Engine's shopper-gap agent counts against it. No AI feature has ever written to a second dataset, mock array, or JSON fixture.

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart LR
    DB["DB[(shoppers table – 60 rows, single source of truth)]"] --> REC["REC[AI Recommendation]"]
    REC --> OUT["OUT[Outreach]"]
    OUT --> TRACK["TRACK[Tracking]"]
    TRACK --> AN["AN[Analytics / AI Operations Engine]"]
    AN --> DB["|reads only, never writes back| DB"]

```

5. API Architecture

Every route below was found in the router source (see `grep audit` in the MVP Scope doc). Full method/path/auth table:

Router (prefix)	Method	Path	Auth	Purpose
auth (/api/auth)	POST	/login	none	Issue bearer token
	GET	/me	bearer	Current user
campaigns (/api/campaigns)	GET	``	bearer	List (bucket filter)
	GET	/ {id}	bearer	Detail
	GET	/ {id} /shops \	/shoppers \	/outreach \
	GET	/ {id} /shops / {id} /recommendatio ns	bearer	AI matching
	POST	/ {id} /shops / {id} /recommendatio ns /approve	bearer	Create invitations
shops (/api/shops)	GET	`` \	/ {id} \	/ {id} /recommenda tions

Router (prefix)	Method	Path	Auth	Purpose
shoppers (/api/shoppers)	GET	""\	/{{id}}\	/{{id}}/campaign-history
recommendations (/api/recommendations)	GET	""	bearer	Cross-campaign recommendation feed
invitations (/api/invitations)	POST	""	bearer	Create invitation
	GET	""\	/{{id}}\	/{{id}}/email
	POST	/{{id}}/simulate\	/send\	/send-test\
email-templates (/api/email-templates)	GET/POST/PUT/DELETE, /{{id}}/duplicate	bearer	CRUD	
tracking	GET	/r/{{token}}	none (public, token-gated)	Click redirect
	GET	/track/open/{{token}}.gif	none	Open pixel
	GET	/api/public/sample-invitation\	/api/public/invitations/{{token}}	none
	POST	/api/invitations/{{token}}/respond	none (token-gated)	Accept/decline
	GET	/api/tracking/summary\	/api/tracking/events	bearer
notifications (/api/notifications)	GET	""	bearer	Recent event feed
integrations (/api/integrations)	GET	""\	/sync-logs\	/{{provider}}
	PUT	/{{provider}}/config	bearer	Update config/secrets
	POST	/{{provider}}/test\	/email/test-send\	/sassie/sync
webhooks (/api/webhooks)	POST	/sendgrid	signature-verified (optional key)	Delivery/open/click/bounce events
ai (/api/ai)	POST	/parse-requirements	bearer	Requirement parser
	GET	/acceptance-probability	bearer	Acceptance predictor
	GET	/campaigns/{{id}}/shops/{{id}}/outreach-priority	bearer	Outreach prioritization
	GET	/campaigns/{{id}}/health\	/performance	bearer
	POST	/campaigns/{{id}}/optimize-assignments	bearer	Assignment optimizer
	GET	/anomalies\	/data-quality	bearer
	GET	/campaigns/{{id}}/feedback-analysis	bearer	Report/sentiment/QA
	POST	/ask	bearer	NL insights / Operations Assistant
	GET	/next-best-actions\	/action-center	bearer

Router (prefix)	Method	Path	Auth	Purpose
	POST	/personalize-email	bearer	Email personalization
dashboard (/api/dashboard)	GET	/metrics	bearer	Dashboard KPIs
misc (/api)	GET	/audit-logs \	/insights \	/settings

No endpoint in the above list is PROPOSED — every one is live in the current router files. There is no rate-limited/paginated cursor API (list endpoints return full arrays), and there is no GraphQL layer.

6. Integration Architecture

Integration	Purpose	Direction	Auth	Status (verified)	Failure handling
SASSIE	Shopper/campaign data sync	Inbound (pull)	API key/client ID (optional)	Demo adapter (services/integrations/sassie.py) always usable; real API path exists in code but no real credentials configured — status reported as DEMO, never fake CONNECTED	sync_logs records failed/partial runs; Operations Engine surfaces staleness via integration_awareness.py
SendGrid	Outbound email delivery	Outbound	API key	Code path fully implemented (_send_via_sendgrid); EMAIL_PROVIDER defaults to mock (no external call) unless a real key is set	Outbox worker retries up to EMAIL_MAX_ATTEMPTS (3); webhook records bounces/failures
SendGrid Event Webhook	Delivery/open/click/bounce events	Inbound	Optional ECDSA signature verification key	Implemented (routers/webhooks.py)	Unverified requests accepted if no key configured (documented as demo-only)
Google Maps	Geocoding/distance	Outbound	API key	Config/test UI exists; no live geocoding call is used by matching — haversine_km (pure coordinate math) is what recommendations actually use	N/A — falls back to coordinate math automatically
SMS provider	Outreach via SMS	Outbound	API key	Config UI only, no send code path	N/A
AI provider (OpenAI/Gemini/etc.)	External AI model	Outbound	API key	Not used anywhere — services/semantic_matching.py explicitly does not call any external AI API; AI_PROVIDER/AI_API_KEY env vars exist only so a future swap is a config change	N/A
PostgreSQL	Primary datastore (production path)	—	connection string	Supported (docker-compose provisions it); SQLite used in this dev environment	Alembic migrations target both

Integration	Purpose	Direction	Auth	Status (verified)	Failure handling
Vector database	Embedding storage/search	—	—	Not present — TF-cosine is computed in-process, no vector index needed at 60-row scale	N/A
Redis / Celery	Background job queue	—	—	Not present — <code>services/outbox.py</code> implements an in-process asyncio background task instead	N/A

7. Deliberate Non-Implementations (and why)

These were requested in various rounds of scope but intentionally not built, to avoid unused infrastructure in a 60-record demo:

Item	Reason not built
Real sentence-embedding model / vector DB	TF-cosine similarity over 60 shopper profiles is sub-millisecond; a vector index would add infra with no measurable benefit at this scale. The <code>embed()</code> function boundary is intentionally isolated so this can change later without touching any caller.
Embedding/recommendation caching	Same reasoning — recomputing 60 scores per request is already cheap; a cache would add invalidation complexity for no latency win.
Neo4j graph layer	No regional-density/relationship query has been requested that a SQL query can't already answer at this data volume.
Kafka / event bus	No consumer currently needs to react to <code>SHOPPER_UPDATED</code> -style events asynchronously; the existing outbox worker + on-demand computation cover every current use case.
LangGraph / CrewAI multi-agent framework	The Operations Engine already separates responsibilities into 5 distinct agent functions with a clear <code>detect→analyze→recommend→approve</code> flow. Adding an orchestration framework wouldn't change behavior — the explicit instruction was not to add one "merely for branding."
Multi-role RBAC enforcement	<code>users.role</code> column exists but no route branches on it; only one admin account is used throughout the demo. Real RBAC is a proposed Phase 4 item.

SHOPPERMATCH AI WORKFLOWS

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI — Workflows

All workflows below trace to the actual endpoints/functions listed in `SHOPPERMATCH_AI_ARCHITECTURE.md`. No step is aspirational unless explicitly marked (**proposed**).

1. Complete End-to-End Workflow

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
graph TD
    A[Admin: Create Campaign] --> B[Define requirements as free text]
    B --> C["AI Requirement Parser\n(services/ai/requirement_parser.py)"]
    C --> D[Structured filters saved to campaigns.parsed_requirements]
    A --> E[Add Shops to campaign]
    D --> F["Hard Eligibility Filtering\n(active, availability, radius, min rating/completion, categories)"]
    E --> F
    F --> G["(shoppers table - 60 rows)"]
    G --> H["Semantic + Structured Scoring\n(services/semantic_matching.py)"]
    H --> I[Ranked candidates: match_score, confidence, breakdown, reasons]
    I --> J[Human reviews Top 5/10/All shortlist]
    J --> K{Approve?}
    K -->|Yes| L["POST recommendations/approve\ncreates Invitation rows"]
    K -->|No - expand radius / view more| J
    L --> M[Outreach composer: pick template or Generate with AI]
    M --> N["EmailComposition or default template\nrendered by services/email.py"]
    N --> O[Tracking token + assignment link + open pixel injected]
    O --> P{EMAIL_PROVIDER}
    P -->|sendgrid| Q[SendGrid HTTP API]
    P -->|mock/smtp/emailjs/direct| Q
    Q --> R[Shopper receives email]
    R --> S["Shopper opens email\n /track/open/token.gif"]
    S --> T[invitation_events: email_opened]
    R --> U["Shopper clicks assignment link\n /r/token"]
    U --> V[invitation_events: link_clicked]
    U --> W[Redirect to /invite/token landing page]
    W --> X{Accept or Decline}
    X --> Y["POST /api/invitations/token/respond"]
    Y --> Z[invitation_events: assignment_accepted/declined]
    Z --> AA[Campaign status/progress recomputed from real invitation data]
    AA --> AB["AI Campaign Health / Performance\n(services/ai/campaign_predictor.py)"]
    AB --> AC["AI Operations Engine\n5 agents -> Action Center"]
    AC --> AD[Human approves next action]
```

2. Campaign Workflow by Bucket

Buckets are computed by `routers/campaigns.py::status_bucket()` from `campaign.status`, not a separate field.

Active campaign

Code

```
Campaign (status=active)
→ Shops (with required_shoppers, coordinates, category)
→ AI Recommendations (run per shop, on demand)
→ Candidate shortlist → human approval → Invitation rows created
→ Outreach (template or AI-personalized) → SendGrid/mock
→ Tracking (open/click/response events)
→ Progress (completed_shops / total_shops, shown on Overview + Dashboard)
→ AI Campaign Health (readiness %, shop coverage, eligible shoppers, candidate quality, expected completion, risks)
```

UI: `CampaignDetail.tsx` — Overview, Shops, Shoppers, AI Recommendations, Outreach, Tracking, Insights, Audit Logs tabs. Backend: `routers/campaigns.py` + `routers/ai.py`. DB: `campaigns`, `shops`, `invitations`, `invitation_events`.

Upcoming campaign

Code

```
Campaign (status=upcoming, deadline in the future)
→ AI Campaign Readiness (same campaign_health() function as Active – one function, this presentation)
→ Eligible shopper coverage vs. required count
→ Risks surfaced (e.g. "Pune currently has insufficient eligible shoppers for N shop(s)")
→ Recruitment/outreach preparation via the same Recommendations/Outreach tabs
```

Same backend function (`campaign_predictor.campaign_health`) as Active — the "readiness" framing is just how the UI presents the identical computation for a not-yet-started campaign. No separate readiness-only code path exists.

Completed campaign

Code

```
Campaign (status=completed)
→ AI Performance Summary (services/ai/campaign_predictor.py::performance_summary)
→ Completion rate, response rate, accepted/declined counts
→ Per-city acceptance rate comparison
→ Templated (not model-generated) summary sentence, e.g.
  "Acceptance was strongest in Mumbai (78%) and lower in Pune (52%)."
```

UI: `AiPerformanceCard` in the Insights tab, shown only when `bucket === "completed"`.

3. Shopper Management Workflow

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart TD
    DB[(DB)] --> PROFILE[Shopper Profile:\nname, email, phone, city/state/zip, lat/lng]
    DB --> AVAIL[availability_status]
    DB --> EXP[experience_description, skills, years_experience, certifications]
    DB --> LOC[latitude/longitude, preferred_distance_km, preferred_locations]
    DB --> CAT[categories, preferred_categories]
    DB --> RATE[rating]
    DB --> HIST[completion_rate, previous_assignments, previous_clients]
    PROFILE --> MATCH1[MATCH[AI Matching reads every field above]]
    AVAIL --> MATCH2[MATCH]
```

```

EXP --> MATCH
LOC --> MATCH
CAT --> MATCH
RATE --> MATCH
HIST --> MATCH
MATCH --> CAMPHIST["Shopper's own campaign history\n(GET /api/shoppers/id/campaign-history)"]

```

Every one of these fields feeds directly into `score_shopper()`'s seven weighted factors (semantic similarity, distance, category experience, availability, completion history, rating, client experience) — see Architecture doc §1. Fields the product brief mentions but the schema doesn't actually have (e.g. a separate "response_rate" column) are **never fabricated**: `score_shopper` only reads columns that exist, and the acceptance predictor computes response/acceptance rate live from invitations rather than storing a stale duplicate.

4. Outreach Architecture (detailed)

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    SEL1[Select Campaign] --> SEL2[Select Shop]
    SEL2 --> SEL3[Select Shopper]
    SEL3 --> SEL4{Template or AI?}
    SEL4 -->|Template| TPL["Load EmailTemplate row\n(GET /api/email-templates)"]
    SEL4 -->|"Generate with AI"| AI["POST /api/ai/personalize-email\n\n(services/ai/email_personalizer.py)"]
    TPL --> CTX["services/email.py::build_variable_context()"]
    AI --> CTX2["AI-composed subject/body – still passes through the same renderer"]
    CTX --> RENDER["render_variables() / render_composed_email()"]
    CTX2 --> RENDER
    RENDER --> PREVIEW["Preview in composer – user can edit"]
    PREVIEW --> GEN["POST /api/invitations\n\n creates Invitation row + tracking_token"]
    GEN --> LINK["build_click_url() → /r/token?utm_*"]
    LINK --> PIXEL["pixel_url() → /track/open/token.gif"]
    PIXEL --> HTML["Final HTML assembled"]
    HTML --> SEND["POST /{id}/send → services/outbox.py queues EmailJob"]
    SEND --> PROVIDER["send_email() dispatches to configured EMAIL_PROVIDER"]
    PROVIDER --> EVENTS["invitation_events: email_sent/delivered/opened/clicked"]
    EVENTS --> RESPONSE["Shopper accept/decline"]
    RESPONSE --> AUDIT["record_audit() – every send/generate action logged"]

```

Retry/failure: `services/outbox.py` retries a queued `EmailJob` up to `EMAIL_MAX_ATTEMPTS` (default 3) with `next_attempt_at` backoff; failures are recorded on the job row (`last_error`) and as an `invitation_events` entry, never silently dropped.

Never duplicated: both the manual Template flow and the AI-generated flow converge on the exact same `services/email.py` rendering functions before a message is ever queued — there is one rendering code path, not two.

5. Tracking / Response Workflow

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

sequenceDiagram
    participant Shopper
    participant Email
    participant FastAPI
    participant DB as invitation_events

```

```

Email->>FastAPI: GET /track/open/{token}.gif (pixel loads)
FastAPI->>DB: record email_opened
Shopper->>Email: clicks "View Assignment"
Email->>FastAPI: GET /r/{token}?utm_*
FastAPI->>DB: record link_clicked
FastAPI->>Shopper: 302 redirect to /invite/{token}
Shopper->>FastAPI: GET /api/public/invitations/{token}
FastAPI->>Shopper: campaign/shop/compensation/deadline (read-only, token-gated)
Shopper->>FastAPI: POST /api/invitations/{token}/respond {accepted|declined, note?}
FastAPI->>DB: record assignment_accepted/declined
FastAPI->>Shopper: confirmation

```

Privacy/security notes: the tracking token is a random UUID, never a raw database ID; there is no PII in the URL besides the token itself; the public endpoints only ever return the specific invitation matching that exact token (no enumeration).

6. AI Operations Engine Flow (Phase 23/24)

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```

flowchart TD
    MON[Monitor every active/upcoming campaign] --> AGENTS
    subgraph AGENTS[5 agents - services/ai/operations_engine.py]
        COV[Coverage Agent]
        OUT[Outreach Agent]
        DL[Deadline Agent]
        SHP[Shopper/Recruitment Agent]
        CAMP[Campaign Agent]
    end
    AGENTS --> ISSUES["Each agent emits:\ncondition, severity, reason,\nrecommended_action, action_type"]
    ISSUES --> RISK["required_approval computed\n(view/prepare = low risk, assign/send/change = high risk)"]
    RISK --> AGG["Aggregator (campaign_action_items):\nnone top issue per campaign, or on_track"]
    AGG --> UI[AI Action Center - Dashboard]
    UI --> HUMAN{Human reviews}
    HUMAN -->|View Candidates / Generate Follow-up| PREPARE[Low-risk: opens the relevant tab, prepares a draft]
    HUMAN -->|Approve| EXECUTE["High-risk: goes through the SAME existing\napprove-gated endpoints (recommendations/approve,\noptimize-assignments)"]
    EXECUTE --> AUDIT[record_audit logs the action]

```

This is deliberately **not** an autonomous agent loop — every "Execute" step in the diagram is a human clicking an existing button; the Operations Engine only ever **proposes**.

7. Demo Scenario Script (Section 28)

A recommended order for a live client walkthrough, matching what's actually built:

1. **Login** (/login, demo credentials shown on screen).
2. **Dashboard** — point out the funnel KPIs, the AI Action Center (real severity-ranked issues), and ask the Operations Assistant a question live (e.g. "Which campaign needs attention?").
3. **Open an Active Campaign → AI Recommendations tab.**
4. **Run AI Matching** for a shop — show the requirement summary sentence, the eligible/excluded breakdown, and the Top 5/10/All shortlist toggle.

5. **View Match Breakdown** on a candidate — show the weighted score table, confidence label, and the AI Acceptance Probability panel (point out it says "Insufficient historical data" when honest, not a fake number).
 6. **View Profile** on a candidate — opens the `ShopperDrawer` with real rating/completion/campaign history.
 7. **Approve** a shortlist selection — show the resulting invitation reference IDs.
 8. **AI Outreach Prioritization card** — explain HIGH/MEDIUM/LOW.
 9. Go to **Outreach** — select the same campaign/shop/shopper, click "**Generate with AI**" — show the personalized draft populate live, then explain the user must still review/edit before sending.
 10. **Send** (or **Send Test** to a real inbox) — show the tracking token/link get generated.
 11. Open the received email (if using Send Test) → click the assignment link → land on the shopper page → **Accept**.
 12. Return to **Tracking** — show the event moved from clicked → accepted in real time.
 13. Open **Insights** — show the Data Quality and Anomaly cards, and ask the AI a natural-language question.
 14. Return to **Dashboard Action Center** — show the campaign's status changed now that a real assignment exists.
-

8. Testing Strategy (verified vs. proposed)

Verified: `backend/tests/test_acceptance.py` exists (pytest + pytest-asyncio + httpx are in `requirements.txt`), exercising the accept/decline endpoint. No other test files exist in the repository.

Proposed E2E test (not currently implemented — this is the target scenario a future test suite should cover):

Code

```
1. Seed/verify 60 shoppers exist
2. Create a campaign + shop via API
3. Call GET /shops/{id}/recommendations – assert ranked results, all shopper_ids trace to real rows
4. Call POST recommendations/approve – assert Invitation rows created
5. Call POST /api/ai/personalize-email – assert subject/body reference only real fields
6. Call POST /api/invitations/{id}/send – assert EmailJob queued
7. Simulate open/click via the tracking endpoints
8. Call POST /respond – assert invitation_events + campaign progress update
9. Call GET /api/ai/campaigns/{id}/performance – assert numbers match the invitations created above
```

Status: this end-to-end test does not exist yet — flagged here as a Phase 4 backlog item, not claimed as implemented.

SHOPPERMATCH AI EMAIL OUTREACH ARCHITECTURE

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI — Email Templates & Outreach Architecture

This document answers the explicit brief: Email Templates must not be an isolated UI feature — it must be interconnected with Outreach through one reusable backend rendering service. **Verified finding: this is already true in the current codebase.** `services/email.py` is that one shared service; both the Email Templates CRUD and the Outreach composer's AI-generation path render through it. Nothing here needed to be built to satisfy the "no duplicated logic" requirement — this document describes what exists and calls out the gaps precisely.

1. Architecture (as implemented)

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
flowchart TD
    TPL["email_templates table\n(name, subject, html_body, active)"] --> RENDER
    COMP["email_compositions table\n(per-invitation hand-edited override)"] --> RENDER
    AIGEN["services/ai/email_personalizer.py\n(AI-generated subject/body)"] --> RENDER
    RENDER["services/email.py\nbuild_variable_context / render_variables /\nrender_composed_email / render_email"]
    RENDER --> CAMPAIGN[(campaigns)]
    RENDER --> SHOP[(shops)]
    RENDER --> SHOPPER[(shoppers)]
    RENDER --> LINK["build_click_url() → /r/token"]
    RENDER --> PIXEL["pixel_url() → /track/open/token.gif"]
    LINK --> HTML[Final email HTML]
    PIXEL --> HTML
    HTML --> SENDFN["send_email()"]
    SENDFN --> SENDGRID[SendGrid / SMTP / EmailJS / direct-MTA / mock]
    SENDGRID --> EVENTS[(invitation_events)]
```

One rendering service, two callers, zero duplicated logic — confirmed by reading `services/email.py`: `build_variable_context()` and `render_variables()` are called identically whether the subject/body came from a saved `EmailTemplate`, a hand-edited `EmailComposition`, or an AI-generated draft from `email_personalizer.py`. The AI generation endpoint (POST `/api/ai/personalize-email`) returns `{subject, body}` text which the Outreach composer loads into the **same** editable state (`setSubject/setBody`) used for template-based drafts — it does not bypass the renderer or the review step.

2. Email Template Module — current state vs. requested fields

Requested field	In `EmailTemplate` model today?	Status
name	✓	Implemented

Requested field	In `EmailTemplate` model today?	Status
subject	✓	Implemented
html_body	✓	Implemented
active	✓(boolean)	Implemented
created_at / updated_at	✓	Implemented
text_body (plain-text alternative)	✗	Pending
category	✗	Pending
description	✗	Pending
created_by	✗no user attribution on templates)	Pending
is_default	✗no "set as default template" concept)	Pending

Requested actions vs. implemented endpoints:

Action	Endpoint	Status
Create	POST /api/email-templates	
Edit	PUT /api/email-templates/{id}	
Delete	DELETE /api/email-templates/{id}	
Duplicate	POST /api/email-templates/{id}/duplicate	
List	GET /api/email-templates	
Save (from Outreach composer, "Save as Template")	Uses the same POST /api/email-templates	
Preview	No dedicated template-preview endpoint; preview happens client-side after variable substitution in the Outreach composer, and server-side via GET /api/invitations/{id}/email?preview=true once an invitation exists	Partial — works at the invitation level, not as a standalone "preview this template" action
Test Email	POST /api/invitations/{id}/send-test (invitation-scoped) and POST /api/integrations/email/test-send (generic connectivity test) — no *template-scoped* test-send	Partial
Use Template	Selecting it in the Outreach composer's Template dropdown	
Set Default Template	No such concept exists	Pending

Variable tokens

Requested vs. actually supported (from `services/email.py::VARIABLE_TOKENS` and `build_variable_context()`):

Requested token	Actual implementation
{{shopper_name}}	✓exact match (first name only)
{{shop_name}}	✓exact match
{{campaign_name}}	✓exact match

Requested token	Actual implementation
{{location}}	✓exact match (city, state)
{{compensation}}	✓exact match (formatted with currency symbol)
{{deadline}}	✓present, but actually renders the shop's visit window (<code>_fmt_window(visit_start, visit_end)</code>), not the campaign deadline field
{{visit_window}}	✗not a separate token — folded into {{deadline}} above
{{assignment_link}}	✓exact match (click-tracking URL with UTM params)
{{client_name}}	✓exact match
{{invitation_id}}	✓present but not in the original request list — maps to the human-readable reference (e.g. INV-0007)

Unknown `{{token}}` patterns are deliberately left as literal text in the output (not silently dropped) so a typo is visible in preview rather than producing blank text — verified in `render_variables()`.

3. Email Send Workflow (as implemented)

Diagram (Mermaid source — flowchart described below; not natively renderable in Word)

```
sequenceDiagram
    participant U as Admin (Outreach.tsx)
    participant API as FastAPI
    participant EmailSvc as services/email.py
    participant DB
    participant Outbox as services/outbox.py
    participant Provider as SendGrid/SMTP/EmailJS/mock

    U->>API: select campaign, shop, shopper, template (or AI-generate)
    API->>EmailSvc: build_variable_context(invitation-in-progress)
    EmailSvc-->>U: rendered preview (subject + HTML)
    U->>API: POST /api/invitations {campaign_id, shop_id, shopper_id, subject, body}
    API->>DB: INSERT invitations (tracking_token = uuid4(), status=created)
    API->>DB: INSERT invitation_events (invitation_created)
    U->>API: POST /api/invitations/{id}/send
    API->>Outbox: enqueue EmailJob(status=queued)
    Outbox->>EmailSvc: render_invitation_email(inv)
    EmailSvc->>EmailSvc: inject tracking pixel + click-tracking link
    Outbox->>Provider: send_email(rendered message)
    Provider-->>Outbox: success/failure
    Outbox->>DB: UPDATE email_jobs.status, invitations.sent_at
    Outbox->>DB: INSERT invitation_events (email_sent or email_failed)
    Provider-->>API: (async) webhook – delivered/opened/clicked/bounced
    API->>DB: INSERT invitation_events (per webhook event)
```

Every step above is a real code path (`routers/invitations.py`, `services/email.py`, `services/outbox.py`, `routers/webhooks.py`) — none is proposed.

4. Email Tracking Architecture (Section 14)

Concept	Implementation	Table/field
Tracking Token	uuid4() generated per invitation, never a raw DB id	invitations.tracking_token
Tracking URL	/r/{token}?utm_source=...	Built by build_click_url()
Tracking Pixel	/track/open/{token}.gif — 1x1 transparent gif, no JS required	Built by pixel_url(), served by routers/tracking.py
Invitation	One row per (campaign, shop, shopper) outreach attempt	invitations
Invitation Event	Immutable append-only log	invitation_events

Events actually recorded (from `models.py::EventType`): `invitation_created`, `email_queued`, `email_sent`, `email_delivered`, `email_opened`, `link_clicked`, `assignment_accepted`, `assignment_declined`, `email_bounced`, `email_failed`, `email_deferred`. This is a superset of the requested `sent/delivered/opened/clicked/accepted/declined` list — `bounce/failure/deferral` are also tracked.

Rates calculated (Dashboard + Tracking pages, ``routers/dashboard.py``):

Code

```
open_rate      = opened / delivered
click_rate     = clicked / opened
acceptance_rate = accepted / clicked
response_rate  = (accepted + declined) / sent
```

All computed live from `invitation_events/invitations` rows at request time — never fabricated, never cached as a stale stored percentage.

Privacy/security: the click and pixel endpoints only ever look up the exact token in the URL (no enumeration or listing), tokens are UUIDv4 (not sequential/guessable), and no personal data beyond what the invitation already needed is embedded in the URL.

5. Outreach Architecture Summary

Code

Campaign → Shop → Shopper → Template (or AI) → Invitation → Tracking Token → Email → SendGrid → Events → Response

Capability	Status
Email generation (template)	
Email generation (AI-personalized)	
Email preview	(composer-level); (no template-only preview endpoint)
Template selection	
Personalization (variable substitution)	
Tracking (pixel + click)	
Sending (multi-provider)	
Retry on failure	(services/outbox.py, up to 3 attempts)
Failure surfacing	(email_jobs.last_error, invitation_events)

Capability	Status
Audit logging	(<code>record_audit()</code> on AI-generated actions; invitation lifecycle also logged via <code>invitation_events</code>)
Send test email	(invitation-scoped and generic connectivity test)
Follow-up generation	(<code>POST /{id}/follow-up</code> , and AI Action Center's "Generate Follow-up" for opened-not-accepted campaigns)

6. What Is Genuinely Proposed (not implemented)

- `EmailTemplate.text_body`, `.category`, `.description`, `.created_by`, `.is_default` columns — would need a new Alembic migration (`ALTER TABLE email_templates ADD COLUMN ...`), following the existing safe pattern (backup DB file → migrate → verify row counts).
- A template-scoped preview/test-send endpoint (currently only invitation-scoped).
- Per-client default template selection.

None of these gaps block the current end-to-end outreach flow — they are refinements, not missing critical paths.

SHOPPERMATCH AI TOOLS AND COST

ShopperMatch.AI — Documentation Package

SHOPPERMATCH.AI — Tools, Services, API Keys & Cost Estimation ("Tab 5")

Pricing caveat (read first): my knowledge has a cutoff and this project's "today" is 2026-08-13 — provider pricing changes without notice. Every number below marked with ☐ is an approximation from general knowledge, **not a live lookup**, and must be verified against the provider's current official pricing page before it's used in a client quote. Where I have no reasonable confidence at all, I've written **"Pricing requires verification from official provider"** instead of guessing, per instruction.

1. What's actually configured today (verified from `backend/app/config.py`)

Env var	Currently set in this environment?
DATABASE_URL	Defaults to local SQLite file — no hosting cost today
EMAIL_PROVIDER	Defaults to mock — no external email cost today
SENDGRID_API_KEY	Not set in this demo
GOOGLE_MAPS_API_KEY	Not set
SASSIE_API_KEY / SASSIE_API_BASE_URL	Not set — running on the built-in demo adapter
AI_PROVIDER / AI_API_KEY	Not set — no external AI provider is called anywhere in this codebase
SMS_PROVIDER / SMS_API_KEY	Not set — no SMS send code path exists

Bottom line: this demo currently runs at \$0 external API cost. Everything below is what production would add, tool by tool.

2. Tool-by-tool breakdown

SendGrid (email delivery)

- Why needed:** outbound outreach email delivery in production (`EMAIL_PROVIDER=sendgrid`).
- Feature that uses it:** Outreach send, `services/email.py::_send_via_sendgrid`.
- Free tier:** SendGrid has historically offered a limited free/trial tier for low email volume. ☐ Confirm current free-tier email/day limit on sendgrid.com/pricing — this has changed multiple times.
- Paid tier needed for:** any real production sending volume beyond the free tier.

5. **Approximate pricing:** ☐ Entry paid plans have historically been in the low tens of USD/month for a few tens of thousands of emails. **Pricing requires verification from official provider** (sendgrid.com/pricing) for exact current tiers.
6. **Usage-based:** yes, tiered by monthly email volume.
7. **Mandatory:** only if real email delivery is required; the app runs fully functionally on `EMAIL_PROVIDER=mock` for demos.
8. **Free alternative:** the app also supports `smtp` (e.g. Gmail app password, free but low-volume and easily rate-limited) and `emailjs` (EmailJS free tier exists, low volume) as already-coded alternatives — no code change needed to switch, only env vars.
9. **Required for MVP:** no (mock is sufficient for demoing).
10. **Required for production:** yes, if real shoppers must receive real email.

Google Gemini / OpenAI / Anthropic (external AI)

1. **Why needed:** not needed by this codebase today — flagged only because the brief asked for it to be evaluated.
2. **Feature that uses it: none.** Every AI feature in `services/ai/*.py` is local, deterministic Python (regex parsing, TF-cosine similarity, Bayesian smoothing, lexicon sentiment). Confirmed by reading every file in `backend/app/services/ai/`.
3. **Free tier:** each provider has historically offered some free/trial credit. ☐ Verify current terms directly — these change frequently and vary by provider (OpenAI, Google Gemini, Anthropic).
4. **Paid tier needed for:** only if a future upgrade path replaces the TF-cosine matching with real embeddings or adds generative summarization calls.
5. **Approximate pricing:** token-based, varies enormously by model tier. **Pricing requires verification from official provider** for all three (openai.com/pricing, ai.google.dev/pricing, anthropic.com/pricing) — do not quote a number without checking.
6. **Usage-based:** yes, per-token for all three.
7. **Mandatory:** no — the current AI layer has zero dependency on any of these.
8. **Free alternative:** yes — a locally-hosted open-weights embedding model (e.g. via sentence-transformers) would avoid per-token cost entirely if real embeddings are ever adopted.
9. **Required for MVP:** no.
10. **Required for production:** only if/when the product roadmap explicitly decides to upgrade matching quality beyond TF-cosine — not a current requirement.

Google Maps

1. **Why needed:** live geocoding/distance/routing.
2. **Feature that uses it:** none currently — `haversine_km` (pure coordinate math) is what distance scoring actually uses; Maps config exists in Integrations but no live API call happens in the matching pipeline.
3. **Free tier:** Google Maps Platform has historically included a monthly free credit. ☐ Verify current amount at developers.google.com/maps/billing-and-pricing.
4. **Paid tier needed for:** production-scale geocoding/routing beyond the free credit.
5. **Approximate pricing:** per-request, tiered by API (Geocoding vs. Distance Matrix vs. Directions differ). **Pricing requires verification from official provider.**
6. **Usage-based:** yes.
7. **Mandatory:** no — coordinate-based distance already works without it, and shoppers/shops already store lat/lng.
8. **Free alternative:** the coordinate-math fallback already in use, or an open geocoding service (e.g. Nominatim/OpenStreetMap, rate-limited but free) if live geocoding of new addresses is ever needed.

9. **Required for MVP:** no.
10. **Required for production:** only if live traffic-aware routing/geocoding of new addresses becomes a real requirement.

PostgreSQL hosting (Render / Railway / equivalent managed Postgres)

1. **Why needed:** the production datastore (docker-compose already provisions this; only hosting is missing).
2. **Feature that uses it:** everything — it's the single source of truth.
3. **Free tier:** most managed Postgres providers (Render, Railway, Supabase, Neon) have offered small free tiers historically. ☐ Free-tier limits (storage, always-on vs. sleep) change often — verify current terms per provider.
4. **Paid tier needed for:** any real production workload with persistence guarantees / backups.
5. **Approximate pricing:** ☐ entry managed-Postgres paid plans have historically started in the single-digit-to-low-tens of USD/month for small instances. **Pricing requires verification from official provider** for exact current numbers (render.com/pricing, railway.app/pricing).
6. **Usage-based:** partially — storage + compute tiers.
7. **Mandatory:** yes for any real deployment (SQLite is dev/demo-only, not concurrent-write-safe for production).
8. **Free alternative:** self-hosted Postgres on a small VM, or SQLite for genuinely single-writer low-volume use.
9. **Required for MVP:** no (SQLite is fine for a demo).
10. **Required for production:** yes.

Render / Railway / Vercel (app hosting)

1. **Why needed:** hosting the FastAPI+React single Docker image (backend) and/or the SPA separately (frontend, if split).
2. **Feature that uses it:** the whole application.
3. **Free tier:** Render/Railway/Vercel have all historically offered free tiers for small workloads with sleep-on-idle behavior. ☐ Verify current limits per provider.
4. **Paid tier needed for:** always-on production hosting.
5. **Approximate pricing:** ☐ small paid web-service tiers have historically started around single-digit-to-low-tens of USD/month. **Pricing requires verification from official provider.**
6. **Usage-based:** partially (compute/bandwidth tiers).
7. **Mandatory:** yes, something must host the app for it to be reachable outside local dev.
8. **Free alternative:** the free tiers above, for demos/staging only.
9. **Required for MVP:** yes, at minimum a free-tier deployment for stakeholder access.
10. **Required for production:** yes, paid tier for reliability.

Redis (background jobs)

1. **Why needed:** not currently needed — `services/outbox.py` already implements the email queue as an in-process asyncio background task, no Redis/Celery involved.
2. **Feature that uses it:** none today.
3. **Free tier:** most managed Redis providers offer a small free tier. ☐ Verify current terms if adopted.
4. **Approximate pricing:** **Pricing requires verification from official provider** if this becomes needed.
5. **Mandatory:** no.
6. **Required for MVP/production:** no — only if job volume outgrows a single-process in-memory queue.

Sentry (monitoring)

1. **Why needed:** production error monitoring — not currently integrated anywhere in the codebase.
2. **Free tier:** Sentry has historically offered a free developer tier with limited event volume. ☐ Verify current limits.
3. **Approximate pricing:** **Pricing requires verification from official provider** (sentry.io/pricing).
4. **Mandatory:** no, recommended for production.
5. **Required for MVP:** no.
6. **Required for production:** recommended, not currently wired in.

Vector database (pgvector / Pinecone / etc.)

1. **Why needed:** would only matter if real embeddings replace the current TF-cosine approach.
2. **Currently used:** no — no vector database or extension is installed or referenced anywhere in this codebase.
3. **Free alternative:** pgvector is a free, open-source Postgres extension — the cheapest path if this is ever adopted, since Postgres is already the target production database.
4. **Mandatory:** no.
5. **Required for MVP/production:** no, not at 60-shopper scale.

Embedding provider (Sentence Transformers / OpenAI embeddings / etc.)

1. **Currently used:** no.
2. **Free alternative:** self-hosted sentence-transformers model (free, CPU-inference is fine at this record count) instead of a paid embedding API.
3. **Mandatory:** no.

SMS provider (Twilio or equivalent)

1. **Why needed:** SMS outreach channel — mentioned in the roadmap, no send code path exists yet.
2. **Free tier:** most SMS providers offer trial credit, not an ongoing free tier (SMS has a real per-message carrier cost).
3. **Approximate pricing:** per-message, varies heavily by destination country. **Pricing requires verification from official provider** (e.g. twilio.com/pricing) — do not quote a number.
4. **Mandatory:** no.
5. **Required for MVP:** no. **Required for production:** only if SMS outreach is actually built and adopted.

SASSIE (mystery-shopping data platform)

1. **Why needed:** the real upstream source of shopper/campaign data this project is designed to eventually sync from.
2. **Currently used:** demo adapter only (services/integrations/sassie.py) — operates against this project's own existing database, never a real SASSIE call.
3. **Pricing:** SASSIE is an enterprise mystery-shopping platform; **pricing requires verification directly from SASSIE / ISN's existing commercial relationship** — this is not a self-serve SaaS with a public price list.
4. **Mandatory:** only for real production data sync; the demo adapter satisfies every current AI feature without it.

Neo4j

1. **Currently used:** not configured anywhere in this codebase.
2. **Free tier:** Neo4j has historically offered a free Community Edition (self-hosted) and a free tier of Neo4j Aura (managed). ☐ Verify current terms.
3. **Mandatory:** no — see Architecture doc §7 for why this wasn't built.

Kafka

- 1. **Currently used:** not configured anywhere in this codebase.
- 2. **Free alternative:** self-hosted Kafka (free, OSS) or a managed offering (Confluent Cloud has historically had a free trial credit — verify current terms) if ever adopted.
- 3. **Mandatory:** no — see Architecture doc §7.

3. Monthly cost estimate by stage

Cost category	Demo (current state)	Low-scale MVP (real email, small Postgres, small hosting)	Production (real volume, monitoring, all integrations live)
Infrastructure (app hosting)	\$0 (local)	☐ Free—low tens USD/mo (small managed hosting tier)	☐ Tens—low hundreds USD/mo, scales with traffic
Database (Postgres)	\$0 (SQLite file)	☐ Free—low tens USD/mo (small managed instance)	☐ Tens—hundreds USD/mo depending on size/backups
Email (SendGrid or equivalent)	\$0 (mock provider)	☐ Free—low tens USD/mo at low volume	☐ Scales per email volume — verify current tiers
SMS	\$0 (not implemented)	\$0 (not implemented)	☐ Per-message cost if built — verify provider pricing
AI token cost	\$0 (no external AI calls exist)	\$0 (still no external AI calls)	\$0 unless the roadmap explicitly adopts an external embedding/LLM provider
Maps	\$0 (not called)	\$0 (not called)	☐ Per-request if live geocoding is adopted — likely covered by free monthly credit at low volume
Monitoring (Sentry)	\$0 (not integrated)	\$0 (optional free tier if added)	☐ Free—low tens USD/mo depending on event volume
Total (rough order of magnitude)	\$0/mo	☐ roughly free—\$100/mo , dominated by whichever paid tiers are switched on	☐ Cannot be responsibly estimated without current, provider-confirmed numbers and expected volume (campaigns/month, emails/month, shoppers).

I'm not going to put a single confident dollar figure on "production cost" — it depends entirely on email volume, hosting tier, and whether SMS/Maps/AI-provider upgrades are adopted, none of which are fixed yet. The categories above are the right shape for a real quote once those volumes are known; the dollar ranges need re-verification against live pricing pages at quote time.

4. Technology stack table

Technology	Purpose	Where used	Free/Paid	Required for MVP?	Production requirement?
React 18	Frontend UI	frontend/src	Free (OSS)	✓	✓

Technology	Purpose	Where used	Free/Paid	Required for MVP?	Production requirement?
TypeScript	Type-safe frontend	frontend/src/**/*.tsx	Free (OSS)	✓	✓
Tailwind CSS	Styling	frontend/src	Free (OSS)	✓	✓
Recharts	Dashboard charts	Dashboard.tsx	Free (OSS)	✓	✓
Vite	Frontend build tool	build pipeline	Free (OSS)	✓	✓
FastAPI	Backend API framework	backend/app	Free (OSS)	✓	✓
Python 3.12	Backend runtime	backend/.venv	Free (OSS)	✓	✓
SQLAlchemy 2.0 (async)	ORM	backend/app/models.py	Free (OSS)	✓	✓
Alembic	Migrations	backend/alembic	Free (OSS)	✓	✓
SQLite (aiosqlite)	Dev/demo database	default DATABASE_URL	Free	✓(demo)	✗Postgres recommended)
PostgreSQL (asyncpg)	Production database	docker-compose.yml	Free (self-host) / Paid (managed)	✗	✓
httpx	Outbound HTTP (SendGrid/EmailJS/SASSIE)	services/email.py, services/integrations/sassie.py	Free (OSS)	✓	✓
cryptography	Token signing / webhook signature verification	services/security.py, webhooks	Free (OSS)	✓	✓
SendGrid	Real email delivery	services/email.py	Free tier + paid tiers (☐ verify)	✗	✓(or an alternative provider)
Google Maps API	Geocoding (not currently called)	Integrations config only	Free credit + paid (☐ verify)	✗	Optional
pytest / pytest-asyncio / httpx (test)	Testing	backend/tests	Free (OSS)	Recommended	Recommended
Docker	Containerized deployment	Dockerfile, docker-compose.yml	Free (OSS)	✗	Recommended
No vector DB	—	not used	—	✗	✗not needed at current scale)
No Redis/Celery	—	not used	—	✗	✗not needed at current scale)
No external AI provider	—	not used	—	✗	✗current AI layer is fully local)

5. Environment Variables (Section 24)

Only variables actually read by `backend/app/config.py` — **no secret values are included below, only variable names and purpose.**

Variable	Purpose	Required
DATABASE_URL	DB connection string (SQLite or Postgres)	Yes (has a working default)
PUBLIC_BASE_URL	Base URL used to build tracking/pixel links	Yes for correct tracking links
INVITATION_DESTINATION_URL	Optional external post-click redirect	No
SECRET_KEY	Signs access tokens — must change in production	Yes
ENVIRONMENT	development \	production
CORS_ORIGINS	Allowed origins	No (defaults to *)
EMAIL_PROVIDER	mock \	sendgrid \
SENDGRID_API_KEY	SendGrid auth	Only if EMAIL_PROVIDER=sendgrid
SENDGRID_WEBHOOK_VERIFICATION_KEY	Verifies inbound SendGrid webhook signatures	Recommended for production
EMAILJS_SERVICE_ID / EMAILJS_TEMPLATE_ID / EMAILJS_PUBLIC_KEY / EMAILJS_PRIVATE_KEY	EmailJS auth	Only if EMAIL_PROVIDER=emailjs
EMAIL_FROM_NAME / EMAIL_FROM_ADDRESS	Sender identity (also used as SMTP username for Gmail)	Yes
SMTP_HOST / SMTP_PORT / SMTP_USERNAME / SMTP_PASSWORD	SMTP auth	Only if EMAIL_PROVIDER=smtp
EMAIL_WORKER_POLL_SECONDS / EMAIL_MAX_ATTEMPTS	Outbox worker tuning	No (has defaults)
DEMO_ADMIN_NAME / DEMO_ADMIN_EMAIL / DEMO_ADMIN_PASSWORD	Seeded admin login	No (has defaults — must be changed for any real deployment)
ACCESS_TOKEN_EXPIRE_MINUTES	Session length	No (has default)
STATIC_DIR	Built SPA location	No (has default)
AUTO_SEED	Seed demo data if DB empty	No (has default)
TRACKING_RATE_LIMIT_PER_MINUTE	Rate limit on tracking endpoints	No (has default)
DEMO_MODE	Whether unconfigured integrations run on their demo adapter instead of reporting disconnected	No (has default)
SASSIE_API_BASE_URL / SASSIE_API_KEY / SASSIE_CLIENT_ID	Real SASSIE credentials	Only for live SASSIE sync
GOOGLE_MAPS_API_KEY	Google Maps auth	Only if live geocoding is adopted
AI_PROVIDER / AI_MODEL / AI_API_KEY	Reserved for a future external AI provider swap	Not used by any current code path
SMS_PROVIDER / SMS_API_KEY / SMS_SENDER	Reserved for future SMS integration	Not used by any current code path

No `.env` file's actual contents are reproduced anywhere in this documentation — only variable names, matching `.env.example`.

